

Employee Management System

Complete Full-Stack CRUD Documentation

SQL Server • ASP.NET Core Web API • C# • React TypeScript

Prepared as a step-by-step learning document

Table of Contents

1. Project Overview	5
2. Technologies Used	5
Backend	5
Frontend	5
Database	6
3. Full Project Architecture	6
Example:	6
4. SQL Server Setup	6
5. Backend Setup	7
6. Backend Folder Structure	7
7. appsettings.json	7
Explanation	7
8. launchSettings.json	8
Important URL	8
9. Employee Model	9
Explanation	9
10. ApplicationDbContext	9
Explanation	9
11. CreateEmployeeViewModel	10
Explanation	10
12. UpdateEmployeeViewModel	11
Explanation	11
13. Why Use ViewModel?	11
14. IEmployeeService	12
Explanation	12
15. EmployeeService	13
16. EmployeeApiController	15
17. Controller Endpoints	16
18. Service Registration	17
Explanation	17
19. Program.cs	18
Explanation	18
20. EmployeeApi.http	19
Explanation	19
21. Backend Run Command	19
22. Frontend Setup	20
23. Frontend Folder Structure	20
24. .env	21
Important	21

25. config.ts	21
Explanation	21
26. main.tsx	21
Explanation	21
27. App.tsx	22
Explanation	22
28. AppRouter.tsx	22
Explanation	22
29. routes.tsx	22
Routes	22
30. apiTypes.ts	23
Explanation	23
31. employeeZod.ts	23
Explanation	24
32. store.ts	24
Explanation	24
33. baseApi.ts	24
Explanation	24
34. employeeApi.ts	26
Explanation	26
35. apiError.ts	27
Explanation	27
36. format.ts	28
37. Button.tsx	28
38. Input.tsx	29
39. PageHeader.tsx	29
40. StateMessage.tsx	30
41. EmployeeForm.tsx	31
Explanation	32
42. DashboardLayout.tsx	33
Explanation	34
43. Dashboard.tsx	35
Explanation	36
44. NotFound.tsx	36
45. EmployeeList.tsx	37
Explanation	39
46. EmployeeCreate.tsx	39
47. EmployeeEdit.tsx	40
Edit Flow	41
48. index.css	41
49. Tailwind Config	41
50. Frontend Run Command	41

51. Full Working Mechanism	41
Read Employees	41
52. Why Frontend and Backend Both Validate?	43
53. Why Use Service Layer?	43
54. Why Use Dependency Injection?	43
55. Why Use RTK Query?	43
Example:	44
56. Testing Checklist	44
Backend	44
Database	44
Frontend	44
57. Common Errors and Fixes	44
Error: React cannot load employees	44
Error: CORS	45
Error: Cannot connect to database	45
Error: Table does not exist	45
Error: 404 API not found	45
58. Final Summary	45

Employee Management System Documentation

1. Project Overview

This project is a complete Employee CRUD application.

CRUD means:

C = Create employee
R = Read employee list / employee by id
U = Update employee
D = Delete employee

The project has three main parts:

SQL Server Database

ASP.NET Core Web API Backend

React TypeScript Frontend

The full working flow is:

React Frontend

↓
ASP.NET Core Controller
↓
Service Layer
↓
Entity Framework DbContext
↓
SQL Server Database

When the user opens the Employee page in React, React calls the API. The API controller receives the request, calls the service, the service talks to SQL Server using Entity Framework Core, and the result comes back to React.

2. Technologies Used

Backend

ASP.NET Core Web API

C#

Entity Framework Core

SQL Server

Swagger

Dependency Injection

Frontend

React

TypeScript

Vite

React Router

Redux Toolkit Query

React Hook Form

Zod

Tailwind CSS

React Hot Toast

Lucide React Icons

Database

SQL Server

SSMS

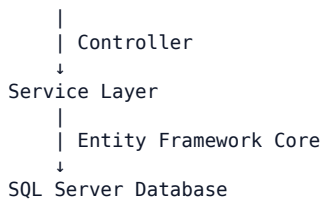
3. Full Project Architecture

Employee Management System

Frontend: React

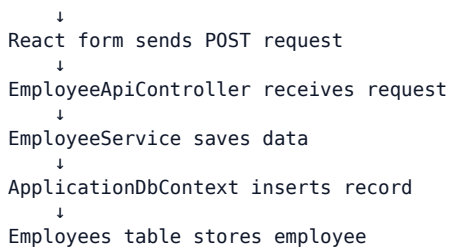


Backend: ASP.NET Core Web API



Example:

User clicks "Add Employee"



4. SQL Server Setup

Open SQL Server Management Studio and run this script.

```
CREATE DATABASE EmployeeApiDb;
GO

USE EmployeeApiDb;
GO

CREATE TABLE Employees
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(150) NOT NULL,
    Email NVARCHAR(255) NOT NULL,
    Salary DECIMAL(18,2) NOT NULL
);
GO
```

To check data:

```
USE EmployeeApiDb;
GO
```

```
SELECT * FROM Employees;
```

Example result:

Id	Name	Email	Salary
1	Munals	munalthapa29@gmail.com	69000.00

5. Backend Setup

Create ASP.NET Core Web API project.

```
dotnet new webapi -n EmployeeApi
cd EmployeeApi
```

Install Entity Framework Core packages:

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

6. Backend Folder Structure

EmployeeApi

```
├── Api
│   └── v1
│       └── EmployeeApiController.cs
├── Data
│   └── ApplicationDbContext.cs
├── Model
│   └── Employee
│       └── Employee.cs
├── Service
│   └── Employee
│       ├── IEmployeeService.cs
│       └── EmployeeService.cs
├── ViewModel
│   └── Employee
│       ├── CreateEmployeeViewModel.cs
│       └── UpdateEmployeeViewModel.cs
├── EmployeeApiServiceRegistrar.cs
├── Program.cs
├── appsettings.json
├── Properties
│   └── launchSettings.json
└── EmployeeApi.http
```

7. appsettings.json

This file stores the database connection string.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=DESKTOP-UD9CMSS\\SQLEXPRESS;Database=EmployeeApiDb;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Explanation

Server=DESKTOP-UD9CMSS\\SQLEXPRESS

This is your SQL Server instance.

Database=EmployeeApiDb

This is the database name.

Trusted_Connection=True

This means Windows Authentication is used.

TrustServerCertificate=True

This is useful for local development.

8. launchSettings.json

This file controls how the backend runs locally.

```
{
  "$schema": "http://json.schemastore.org/launchsettings.json",
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:24550",
      "sslPort": 44388
    }
  },
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5256",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:7043;http://localhost:5256",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Important URL

Your backend runs at:

`http://localhost:5256`

Swagger runs at:

`http://localhost:5256/swagger`

9. Employee Model

File:

Model/Employee/Employee.cs

```
namespace EmployeeApi.Model.Employee
{
    public class Employee
    {
        public int Id { get; set; }

        public string Name { get; set; } = string.Empty;

        public string Email { get; set; } = string.Empty;

        public decimal Salary { get; set; }
    }
}
```

Explanation

This class represents the Employees table.

Employee class	→ Employees table
Id	→ Id column
Name	→ Name column
Email	→ Email column
Salary	→ Salary column

One object of this class represents one row in the table.

Example C# object:

```
new Employee
{
    Id = 1,
    Name = "Munals",
    Email = "munalthapa29@gmail.com",
    Salary = 69000
}
```

Equivalent SQL row:

Id	Name	Email	Salary
1	Munals	munalthapa29@gmail.com	69000

10. ApplicationDbContext

File:

Data/ApplicationDbContext.cs

```
using EmployeeApi.Model.Employee;
using Microsoft.EntityFrameworkCore;

namespace EmployeeApi.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; } = null!;
    }
}
```

Explanation

ApplicationDbContext is the bridge between C# and SQL Server.

This line:

```
public DbSet<Employee> Employees { get; set; } = null!;
```

means:

There is a table called Employees.

When we write:

```
_context.Employees.ToListAsync();
```

Entity Framework generates SQL like:

```
SELECT * FROM Employees;
```

11. CreateEmployeeViewModel

File:

ViewModel/Employee/CreateEmployeeViewModel.cs

```
using System.ComponentModel.DataAnnotations;

namespace EmployeeApi.ViewModel.Employee;

public class CreateEmployeeViewModel
{
    [Required]
    [StringLength(150)]
    public string Name { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    [StringLength(255)]
    public string Email { get; set; } = string.Empty;

    [Range(0, double.MaxValue)]
    public decimal Salary { get; set; }
}
```

Explanation

This class receives data from React or Swagger when creating an employee.

Frontend sends:

```
{
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "salary": 50000
}
```

ASP.NET Core converts that JSON into:

CreateEmployeeViewModel model

Validation attributes:

```
[Required]
```

Field is required.

```
[EmailAddress]
```

Email must be valid.

```
[StringLength(150)]
```

Maximum length is 150 characters.

```
[Range(0, double.MaxValue)]
```

Salary cannot be negative.

12. UpdateEmployeeViewModel

File:

ViewModel/Employee/UpdateEmployeeViewModel.cs

```
namespace EmployeeApi.ViewModel.Employee;

public class UpdateEmployeeViewModel : CreateEmployeeViewModel
{
}
```

Explanation

This class inherits from CreateEmployeeViewModel.

That means it automatically has:

Name

Email

Salary

We use a separate update view model because in real projects, create and update rules can become different later.

13. Why Use ViewModel?

Even though React has forms, users can bypass React using:

Postman

Swagger

Browser DevTools

Custom scripts

Someone could send bad data like:

```
{
  "id": 999,
  "name": "",
  "email": "wrong-email",
  "salary": -5000
}
```

ViewModels protect the backend.

Entity vs ViewModel

Employee Entity

Used for database

CreateEmployeeViewModel

Used for create request

UpdateEmployeeViewModel

Used for update request

This keeps the database model separate from the API request model.

14. IEmployeeService

File:

Service/Employee/IEmployeeService.cs

```
using EmployeeEntity = EmployeeApi.Model.Employee.Employee;

namespace EmployeeApi.Service.Employee
{
    public interface IEmployeeService
    {
        Task<List<EmployeeEntity>> GetAll();

        Task<EmployeeEntity?> GetById(int id);

        Task<EmployeeEntity> Create(EmployeeEntity employee);

        Task<EmployeeEntity> Update(EmployeeEntity employee);

        Task<bool> Delete(int id);
    }
}
```

Explanation

This is an interface.

An interface is like a contract.

It says:

Any employee service must have:

GetAll

GetById

Create

Update

Delete

The controller depends on this interface instead of directly depending on the concrete class.

This is good because later we can replace:

EmployeeService

with:

EmployeeStoredProcedureService

without changing the controller.

15. EmployeeService

File:

Service/Employee/EmployeeService.cs

```
using EmployeeApi.Data;
using Microsoft.EntityFrameworkCore;
using EmployeeEntity = EmployeeApi.Model.Employee.Employee;

namespace EmployeeApi.Service.Employee
{
    public class EmployeeService : IEmployeeService
    {
        private readonly ApplicationDbContext _context;

        public EmployeeService(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<List<EmployeeEntity>> GetAll()
        {
            return await _context.Employees.ToListAsync();
        }

        public async Task<EmployeeEntity?> GetById(int id)
        {
            return await _context.Employees.FindAsync(id);
        }

        public async Task<EmployeeEntity> Create(EmployeeEntity employee)
        {
            _context.Employees.Add(employee);
            await _context.SaveChangesAsync();
            return employee;
        }

        public async Task<EmployeeEntity> Update(EmployeeEntity employee)
        {
            var existingEmployee = await _context.Employees.FindAsync(employee.Id)
                ?? throw new KeyNotFoundException($"Employee {employee.Id} was not found.");

            _context.Entry(existingEmployee).CurrentValues.SetValues(employee);
            await _context.SaveChangesAsync();

            return existingEmployee;
        }

        public async Task<bool> Delete(int id)
        {
            var employee = await _context.Employees.FindAsync(id);

            if (employee is null)
            {
                return false;
            }

            _context.Employees.Remove(employee);
            await _context.SaveChangesAsync();

            return true;
        }
    }
}

GetAll Explanation
return await _context.Employees.ToListAsync();
```

Equivalent SQL:

```
SELECT * FROM Employees;
GetById Explanation
return await _context.Employees.FindAsync(id);
```

Equivalent SQL:

```
SELECT * FROM Employees
WHERE Id = @id;
Create Explanation
_context.Employees.Add(employee);
```

```
await _context.SaveChangesAsync();
```

Equivalent SQL:

```
INSERT INTO Employees
(
    Name,
    Email,
    Salary
)
VALUES
(
    @Name,
    @Email,
    @Salary
);
Update Explanation
var existingEmployee = await _context.Employees.FindAsync(employee.Id);
_context.Entry(existingEmployee).CurrentValues.SetValues(employee);
await _context.SaveChangesAsync();
```

Equivalent SQL:

```
UPDATE Employees
SET
    Name = @Name,
    Email = @Email,
    Salary = @Salary
WHERE Id = @Id;
Delete Explanation
_context.Employees.Remove(employee);
await _context.SaveChangesAsync();
```

Equivalent SQL:

```
DELETE FROM Employees
WHERE Id = @Id;
```

16. EmployeeApiController

File:

Api/v1/EmployeeApiController.cs

```
using EmployeeApi.Service.Employee;
using EmployeeApi.ViewModel.Employee;
using Microsoft.AspNetCore.Mvc;
using EmployeeEntity = EmployeeApi.Model.Employee.Employee;

namespace EmployeeApi.Api.v1
{
    [ApiController]
    [Route("api/employee")]
    public class EmployeeApiController : ControllerBase
    {
        private readonly IEmployeeService _service;

        public EmployeeApiController(IEmployeeService service)
        {
            _service = service;
        }

        [HttpGet]
        public async Task<IActionResult> Get()
        {
            return Ok(await _service.GetAll());
        }

        [HttpGet("{id:int}")]
        public async Task<IActionResult> GetById(int id)
        {
            var employee = await _service.GetById(id);

            return employee is null ? NotFound() : Ok(employee);
        }

        [HttpPost]
        public async Task<IActionResult> Create([FromBody] CreateEmployeeViewModel model)
        {
            var employee = new EmployeeEntity
            {
                Name = model.Name,
                Email = model.Email,
                Salary = model.Salary
            };

            var createdEmployee = await _service.Create(employee);

            return CreatedAtAction(
                nameof(GetById),
                new { id = createdEmployee.Id },
                createdEmployee
            );
        }

        [HttpPut("{id:int}")]
        public async Task<IActionResult> Update(int id, [FromBody] UpdateEmployeeViewModel model)
        {
            if (await _service.GetById(id) is null)
            {
                return NotFound();
            }

            var employee = new EmployeeEntity
            {
                Id = id,
                Name = model.Name,
                Email = model.Email,
                Salary = model.Salary
            };

            return Ok(await _service.Update(employee));
        }

        [HttpDelete("{id:int}")]
        public async Task<IActionResult> Delete(int id)
```

```
        {
            return await _service.Delete(id) ? NoContent() : NotFound();
        }
    }
}
```

17. Controller Endpoints

Get all employees
GET /api/employee

Controller method:

```
[HttpGet]
public async Task<IActionResult> Get()
```

Response:

```
[
  {
    "id": 1,
    "name": "Munals",
    "email": "munalthapa29@gmail.com",
    "salary": 69000
  }
]
Get employee by id
GET /api/employee/1
```

Controller method:

```
[HttpGet("{id:int}")]
public async Task<IActionResult> GetById(int id)
```

Response:

```
{
  "id": 1,
  "name": "Munals",
  "email": "munalthapa29@gmail.com",
  "salary": 69000
}
Create employee
POST /api/employee
```

Request body:

```
{
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "salary": 50000
}
```

Controller method:

```
[HttpPost]
public async Task<IActionResult> Create([FromBody] CreateEmployeeViewModel model)
Update employee
PUT /api/employee/1
```

Request body:

```
{
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "salary": 55000
}
```

Controller method:

```
[HttpPut("{id:int}")]
public async Task<IActionResult> Update(int id, [FromBody] UpdateEmployeeViewModel model)
Delete employee
DELETE /api/employee/1
```

Controller method:

```
[HttpDelete("{id:int}")]
public async Task<IActionResult> Delete(int id)
```

18. Service Registration

File:

EmployeeApiServiceRegistrar.cs

```
using EmployeeApi.Service.Employee;

namespace EmployeeApi;

public static class EmployeeApiServiceRegistrar
{
    public static IServiceCollection AddEmployeeApiServices(this IServiceCollection services)
    {
        services.AddScoped<IEmployeeService, EmployeeService>();

        return services;
    }
}
```

Explanation

This line:

```
services.AddScoped<IEmployeeService, EmployeeService>();
```

means:

Whenever ASP.NET Core sees `IEmployeeService`,
give it `EmployeeService`.

This is called Dependency Injection.

Controller asks for:

`IEmployeeService`

ASP.NET Core gives:

`EmployeeService`

AddScoped Meaning

One `EmployeeService` object is created per HTTP request.

This is recommended when using `ApplicationDbContext`.

19. Program.cs

File:

Program.cs

```
using EmployeeApi;
using EmployeeApi.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddEmployeeApiServices();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("ReactPolicy", policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
    var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    dbContext.Database.EnsureCreated();
}

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

if (!app.Environment.IsDevelopment())
{
    app.UseHttpsRedirection();
}

app.UseCors("ReactPolicy");

app.MapControllers();

app.Run();
```

Explanation

This line registers controllers:

```
builder.Services.AddControllers();
```

This line connects SQL Server:

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));
```

This line registers services:

```
builder.Services.AddEmployeeApiServices();
```

This enables Swagger:

```
builder.Services.AddSwaggerGen();
```

This allows React to call the backend:

```
app.UseCors("ReactPolicy");
```

This maps controller routes:

```
app.MapControllers();
```

20. EmployeeApi.http

File:

EmployeeApi.http

```
@EmployeeApi_HostAddress = http://localhost:5256
```

```
GET {{EmployeeApi_HostAddress}}/api/employee
```

Accept: application/json

###

```
GET {{EmployeeApi_HostAddress}}/api/employee/1
```

Accept: application/json

###

```
POST {{EmployeeApi_HostAddress}}/api/employee
```

Content-Type: application/json

```
{
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "salary": 50000
}
```

###

```
PUT {{EmployeeApi_HostAddress}}/api/employee/1
```

Content-Type: application/json

```
{
  "name": "Jane Doe",
  "email": "jane.doe@example.com",
  "salary": 55000
}
```

###

```
DELETE {{EmployeeApi_HostAddress}}/api/employee/1
```

Explanation

This file is used to test API endpoints directly from Visual Studio or VS Code.

21. Backend Run Command

Run backend:

```
dotnet run
```

Open Swagger:

```
http://localhost:5256/swagger
```

Test:

```
GET    /api/employee
GET    /api/employee/1
POST   /api/employee
PUT    /api/employee/1
DELETE /api/employee/1
```

22. Frontend Setup

Create React project:

```
npm create vite@latest employee-ui -- --template react-ts
cd employee-ui
npm install
```

Install required packages:

```
npm install @reduxjs/toolkit react-redux react-router-dom react-hook-form zod
@hookform/resolvers react-hot-toast lucide-react
```

Install Tailwind CSS:

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

23. Frontend Folder Structure

src

```
├── components
│   ├── common
│   │   ├── PageHeader.tsx
│   │   └── StateMessage.tsx
│   ├── employee
│   │   └── EmployeeForm.tsx
│   ├── layout
│   │   └── DashboardLayout.tsx
│   └── ui
│       ├── Button.tsx
│       └── Input.tsx
├── constants
│   └── config.ts
├── lib
│   ├── apiError.ts
│   └── format.ts
├── pages
│   ├── Dashboard.tsx
│   ├── NotFound.tsx
│   └── employees
│       ├── EmployeeCreate.tsx
│       ├── EmployeeEdit.tsx
│       └── EmployeeList.tsx
├── redux
│   ├── store.ts
│   └── api
│       ├── baseApi.ts
│       └── employeeApi.ts
├── routes
│   ├── AppRouter.tsx
│   └── routes.tsx
├── types
│   └── apiTypes.ts
├── zod
│   └── employeeZod.ts
├── App.tsx
├── main.tsx
└── index.css
```

24. .env

File:

.env

```
VITE_API_URL=http://localhost:5256
```

Important

After creating or changing .env, restart React:

```
npm run dev
```

25. config.ts

File:

src/constants/config.ts

```
export const API_URL = import.meta.env.VITE_API_URL || "http://localhost:5256";
```

Explanation

This stores the backend URL.

If .env has:

```
VITE_API_URL=http://localhost:5256
```

then:

```
API_URL = "http://localhost:5256";
```

26. main.tsx

File:

src/main.tsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { Toaster } from "react-hot-toast";
import { Provider } from "react-redux";
import { BrowserRouter } from "react-router-dom";
import App from "./App";
import "./index.css";
import { store } from "./redux/store";
```

```
createRoot(document.getElementById("root")!).render(
```

```
  <StrictMode>
    <Provider store={store}>
      <BrowserRouter>
        <App />
        <Toaster position="top-right" />
      </BrowserRouter>
    </Provider>
  </StrictMode>,
);
```

Explanation

This is the starting point of React.

```
<Provider store={store}>
```

Gives Redux store to the whole app.

```
<BrowserRouter>
```

Enables routing.

```
<Toaster />
```

Enables toast notifications.

27. App.tsx

File:

src/App.tsx

```
import { AppRouter } from "../routes/AppRouter";

export default function App() {
  return <AppRouter />;
}
```

Explanation

The app only loads routes from AppRouter.

28. AppRouter.tsx

File:

src/routes/AppRouter.tsx

```
import { useRoutes } from "react-router-dom";
import { routes } from "../routes";

export function AppRouter() {
  return useRoutes(routes);
}
```

Explanation

useRoutes reads route definitions from routes.tsx.

29. routes.tsx

File:

src/routes/routes.tsx

```
import { Navigate, type RouteObject } from "react-router-dom";
import { DashboardLayout } from "../components/layout/DashboardLayout";
import Dashboard from "../pages/Dashboard";
import NotFound from "../pages/NotFound";
import EmployeeCreate from "../pages/employees/EmployeeCreate";
import EmployeeEdit from "../pages/employees/EmployeeEdit";
import EmployeeList from "../pages/employees/EmployeeList";

export const routes: RouteObject[] = [
  {
    path: "/",
    element: <DashboardLayout />,
    children: [
      { index: true, element: <Dashboard /> },
      { path: "login", element: <Navigate to="/" replace /> },
      { path: "employees", element: <EmployeeList /> },
      { path: "employees/new", element: <EmployeeCreate /> },
      { path: "employees/:id/edit", element: <EmployeeEdit /> },
      { path: "*", element: <NotFound /> },
    ],
  },
];
```

Routes

/	Dashboard
/employees	Employee list
/employees/new	Create employee
/employees/:id/edit	Edit employee

30. apiTypes.ts

File:

src/types/apiTypes.ts

```
export interface Employee {
  id: number;
  name: string;
  email: string;
  salary: number;
}

export type EmployeePayload = Omit<Employee, "id">;

export interface ApiError {
  status?: number | string;
```

data?:

```
  | string
  | {
    title?: string;
    detail?: string;
    errors?: Record<string, string[]>;
  };
  error?: string;
}
```

Explanation

Employee represents data returned from backend.

```
export type EmployeePayload = Omit<Employee, "id">;
```

This means when creating employee, frontend does not send id.

Create request:

```
{
  "name": "Jane",
  "email": "jane@example.com",
  "salary": 50000
}
```

Response:

```
{
  "id": 1,
  "name": "Jane",
  "email": "jane@example.com",
  "salary": 50000
}
```

31. employeeZod.ts

File:

src/zod/employeeZod.ts

```
import { z } from "zod";

export const employeeSchema = z.object({
  name: z
    .string()
    .trim()
    .min(2, "Name must contain at least 2 characters")
    .max(100),
  email: z
    .string()
    .trim()
    .email("Enter a valid email address")
    .max(150),
  salary: z
```

```
        .number()
        .positive("Salary must be greater than zero"),
    });

    export type EmployeeFormValues = z.infer<typeof employeeSchema>;
```

Explanation

This validates form data before calling API.

Rules:

Name must be at least 2 characters

Email must be valid

Salary must be greater than zero

32. store.ts

File:

src/redux/store.ts

```
import { configureStore } from "@reduxjs/toolkit";
import { useDispatch, useSelector } from "react-redux";
import { baseApi } from "../api/baseApi";

export const store = configureStore({
  reducer: {
    [baseApi.reducerPath]: baseApi.reducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware().concat(baseApi.middleware),
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector = useSelector.withTypes<RootState>();
```

Explanation

This configures Redux store.

RTK Query uses this store to manage API calls, loading states, errors, and cache.

33. baseApi.ts

File:

src/redux/api/baseApi.ts

```
import { createApi, fetchBaseQuery } from "@reduxjs/toolkit/query/react";
import { API_URL } from "../../constants/config";

export const baseApi = createApi({
  reducerPath: "api",
  baseQuery: fetchBaseQuery({
    baseUrl: API_URL,
    headers: { "Content-Type": "application/json" },
  }),
  tagTypes: ["Employee"],
  endpoints: () => ({}),
});
```

Explanation

This is the common API configuration.

If API_URL is:

`http://localhost:5256`

and endpoint is:

`"/api/Employee"`

Final request becomes:

`http://localhost:5256/api/Employee`

ASP.NET Core routing is case-insensitive, so both work:

`/api/Employee`

`/api/employee`

34. employeeApi.ts

File:

src/redux/api/employeeApi.ts

```
import type { Employee, EmployeePayload } from "../../types/apiTypes";
import { baseApi } from "../baseApi";

export const employeeApi = baseApi.injectEndpoints({
  endpoints: (builder) => ({
    getEmployees: builder.query<Employee[], void>({
      query: () => "/api/Employee",
      providesTags: (result) =>
        result
          ? [
              ...result.map(({ id }) => ({ type: "Employee" as const, id })),
              { type: "Employee", id: "LIST" },
            ]
          : [{ type: "Employee", id: "LIST" }],
    }),

    getEmployee: builder.query<Employee, number>({
      query: (id) => `/api/Employee/${id}`,
      providesTags: (_result, _error, id) => [{ type: "Employee", id }],
    }),

    createEmployee: builder.mutation<Employee, EmployeePayload>({
      query: (body) => ({
        url: "/api/Employee",
        method: "POST",
        body,
      }),
      invalidatesTags: [{ type: "Employee", id: "LIST" }],
    }),

    updateEmployee: builder.mutation<Employee, Employee>({
      query: ({ id, ...body }) => ({
        url: `/api/Employee/${id}`,
        method: "PUT",
        body,
      }),
      invalidatesTags: (_result, _error, { id }) => [
        { type: "Employee", id },
        { type: "Employee", id: "LIST" },
      ],
    }),

    deleteEmployee: builder.mutation<void, number>({
      query: (id) => ({
        url: `/api/Employee/${id}`,
        method: "DELETE",
      }),
      invalidatesTags: (_result, _error, id) => [
        { type: "Employee", id },
        { type: "Employee", id: "LIST" },
      ],
    }),
  }),
});

export const {
  useGetEmployeesQuery,
  useGetEmployeeQuery,
  useCreateEmployeeMutation,
  useUpdateEmployeeMutation,
  useDeleteEmployeeMutation,
} = employeeApi;
```

Explanation

This file contains all Employee API calls.

Get Employees
useGetEmployeesQuery()

Calls:

GET <http://localhost:5256/api/Employee>

```
Get Employee By Id
useGetEmployeeQuery(id)
```

Calls:

```
GET http://localhost:5256/api/Employee/1
Create Employee
useCreateEmployeeMutation()
```

Calls:

```
POST http://localhost:5256/api/Employee
Update Employee
useUpdateEmployeeMutation()
```

Calls:

```
PUT http://localhost:5256/api/Employee/1
Delete Employee
useDeleteEmployeeMutation()
```

Calls:

```
DELETE http://localhost:5256/api/Employee/1
Tags Explanation
```

This:

```
invalidatesTags: [{ type: "Employee", id: "LIST" }]
```

means after create, update, or delete, RTK Query reloads the employee list automatically.

35. apiError.ts

File:**src/lib/apiError.ts**

```
import type { ApiError } from "../types/apiTypes";

export function getApiErrorMessage(error: unknown): string {
  const apiError = error as ApiError;

  if (typeof apiError?.data === "string") {
    return apiError.data;
  }

  if (apiError?.data?.errors) {
    const messages = Object.values(apiError.data.errors).flat();

    if (messages.length > 0) {
      return messages.join("\n");
    }
  }

  if (apiError?.data?.detail) {
    return apiError.data.detail;
  }

  if (apiError?.data?.title) {
    return apiError.data.title;
  }

  if (apiError?.error) {
    return apiError.error;
  }

  return "Something went wrong. Please try again.";
}
```

Explanation

This converts API errors into readable messages for toast notifications.

36. format.ts

File:

src/lib/format.ts

```
export function formatCurrency(amount: number) {
  return new Intl.NumberFormat("en-US", {
    style: "currency",
    currency: "USD",
  }).format(amount);
}
```

For Nepal currency, use:

```
export function formatCurrency(amount: number) {
  return new Intl.NumberFormat("en-NP", {
    style: "currency",
    currency: "NPR",
  }).format(amount);
}
```

37. Button.tsx

File:

src/components/ui/Button.tsx

```
import type { ButtonHTMLAttributes, ReactNode } from "react";

type ButtonVariant = "primary" | "secondary" | "danger";

interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {
  children: ReactNode;
  variant?: ButtonVariant;
}
```

```
const styles: Record<ButtonVariant, string> = {
```

primary:

```
  "bg-teal-700 text-white hover:bg-teal-800 focus-visible:ring-teal-500",
```

secondary:

```
  "border border-slate-300 bg-white text-slate-700 hover:bg-slate-50
    focus-visible:ring-slate-400",
```

danger:

```
  "bg-red-600 text-white hover:bg-red-700 focus-visible:ring-red-500",
};
```

```
export function Button({
  children,
  variant = "primary",
  className = "",
  type = "button",
  ...props
}: ButtonProps) {
  return (
    <button
      type={type}
      className={`inline-flex h-10 items-center justify-center gap-2 rounded-lg px-4 text-sm
        font-semibold transition disabled:cursor-not-allowed disabled:opacity-60
        focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-offset-2
        ${styles[variant]} ${className}`}
      {...props}
    >
      {children}
    </button>
  );
}
```

38. Input.tsx

File:

src/components/ui/Input.tsx

```
import { useRef, type InputHTMLAttributes } from "react";

interface InputProps extends InputHTMLAttributes<HTMLInputElement> {
  label: string;
  error?: string;
}

export const Input = forwardRef<HTMLInputElement, InputProps>(
  ({ label, error, className = "", ...props }, ref) => (
    <label className="block space-y-1.5">
      <span className="text-sm font-medium text-slate-700">{label}</span>

      <input
        ref={ref}
        className={`h-11 w-full rounded-lg border bg-white px-3 text-sm text-slate-900
          outline-none transition placeholder:text-slate-400 focus:border-teal-600
          focus:ring-2 focus:ring-teal-100 ${
            error ? "border-red-400" : "border-slate-300"
          } ${className}`}
        {...props}
      />

      {error && <span className="block text-xs text-red-600">{error}</span>}
    </label>
  ),
);

Input.displayName = "Input";
```

39. PageHeader.tsx

File:

src/components/common/PageHeader.tsx

```
import type { ReactNode } from "react";

interface PageHeaderProps {
  title: string;
  description: string;
  action?: ReactNode;
}

export function PageHeader({ title, description, action }: PageHeaderProps) {
  return (
    <div className="flex flex-col gap-4 sm:flex-row sm:items-center sm:justify-between">
      <div>
        <h1 className="text-2xl font-bold tracking-tight text-slate-900">
          {title}
        </h1>

        <p className="mt-1 text-sm text-slate-500">
          {description}
        </p>
      </div>

      {action}
    </div>
  );
}
```

40. StateMessage.tsx

File:

src/components/common/StateMessage.tsx

```
import { AlertCircle, LoaderCircle } from "lucide-react";

export function LoadingState() {
  return (
    <div className="flex min-h-52 items-center justify-center gap-3 text-slate-500">
      <LoaderCircle className="animate-spin" size={22} />
      Loading...
    </div>
  );
}

export function ErrorState({ message }: { message: string }) {
  return (
    <div className="flex min-h-52 flex-col items-center justify-center gap-2 rounded-xl border
      border-red-200 bg-red-50 p-6 text-center text-red-700">
      <AlertCircle size={26} />
      <p className="font-medium">{message}</p>
    </div>
  );
}
```

41. EmployeeForm.tsx

File:

src/components/employee/EmployeeForm.tsx

```
import { zodResolver } from "@hookform/resolvers/zod";
import { ArrowLeft, Save } from "lucide-react";
import { useEffect } from "react";
import { useForm } from "react-hook-form";
import { useNavigate } from "react-router-dom";
import { employeeSchema, type EmployeeFormValues } from "../../zod/employeeZod";
import { Button } from "../../ui/Button";
import { Input } from "../../ui/Input";

interface EmployeeFormProps {
  defaultValues?: EmployeeFormValues;
  isSubmitting: boolean;
  submitLabel: string;
  onSubmit: (values: EmployeeFormValues) => Promise<void>;
}

export function EmployeeForm({
  defaultValues,
  isSubmitting,
  submitLabel,
  onSubmit,
}: EmployeeFormProps) {
  const navigate = useNavigate();

  const {
    register,
    handleSubmit,
    reset,
    formState: { errors },
  } = useForm<EmployeeFormValues>({
    resolver: zodResolver(employeeSchema),
    defaultValues: defaultValues ?? {
      name: "",
      email: "",
      salary: 0,
    },
  });

  useEffect(() => {
    if (defaultValues) {
      reset(defaultValues);
    }
  }, [defaultValues, reset]);

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="space-y-6">
      <div className="grid gap-5 rounded-xl border border-slate-200 bg-white p-5 shadow-sm md:grid-cols-2 md:p-6">
        <Input
          label="Full name"
          placeholder="e.g. Jane Smith"
          error={errors.name?.message}
          {...register("name")}
        />

        <Input
          label="Email address"
          type="email"
          placeholder="jane@example.com"
          error={errors.email?.message}
          {...register("email")}
        />

        <Input
          label="Annual salary"
          type="number"
          step="0.01"
          min="0"
          placeholder="50000"
          error={errors.salary?.message}
          {...register("salary", { valueAsNumber: true })}
        />
      </div>
    </form>
  );
}
```

```
    <div className="flex justify-end gap-3 rounded-xl border border-slate-200 bg-white p-4
      shadow-sm">
      <Button variant="secondary" onClick={() => navigate("/employees")}>
        <ArrowLeft size={17} />
        Cancel
      </Button>

      <Button type="submit" disabled={isSubmitting}>
        <Save size={17} />
        {isSubmitting ? "Saving..." : submitLabel}
      </Button>
    </div>
  </form>
);
}
```

Explanation

This form is reused by:

EmployeeCreate

EmployeeEdit

Create uses empty form.

Edit uses existing employee data.

42. DashboardLayout.tsx

File:

src/components/layout/DashboardLayout.tsx

```
import {
  BriefcaseBusiness,
  LayoutDashboard,
  Menu,
  Users,
  X,
} from "lucide-react";
import { useState } from "react";
import { NavLink, Outlet } from "react-router-dom";

const navigation = [
  { label: "Dashboard", to: "/", icon: LayoutDashboard, end: true },
  { label: "Employees", to: "/employees", icon: Users, end: false },
];

export function DashboardLayout() {
  const [open, setOpen] = useState(false);

  return (
    <div className="min-h-screen bg-slate-50">
      <aside
        className={`fixed inset-y-0 left-0 z-30 w-64 border-r border-slate-200 bg-slate-950
          text-white transition-transform lg:translate-x-0 ${
            open ? "translate-x-0" : "-translate-x-full"
          }`}
      >
        <div className="flex h-16 items-center justify-between border-b border-slate-800 px-5">
          <div className="flex items-center gap-2 font-bold">
            <span className="grid size-9 place-items-center rounded-lg bg-teal-600">
              <BriefcaseBusiness size={19} />
            </span>
            Dorthy Admin
          </div>

          <button
            className="lg:hidden"
            onClick={() => setOpen(false)}
            aria-label="Close menu"
          >
            <X size={21} />
          </button>
        </div>

        <nav className="space-y-1 p-3">
          {navigation.map(({ label, to, icon: Icon, end }) => (
            <NavLink
              key={to}
              to={to}
              end={end}
              onClick={() => setOpen(false)}
              className={({ isActive }) =>
                `flex items-center gap-3 rounded-lg px-3 py-2.5 text-sm font-medium transition
                  ${
                    isActive
                      ? "bg-teal-600 text-white"
                      : "text-slate-300 hover:bg-slate-800 hover:text-white"
                  }`}
            >
              <Icon size={18} />
              {label}
            </NavLink>
          ))}
        </nav>
      </aside>

      {open && (
        <button
          className="fixed inset-0 z-20 bg-slate-950/50 lg:hidden"
          onClick={() => setOpen(false)}
          aria-label="Close menu overlay"
        />
      )}
    </div>
  );
}
```

```
<div className="lg:pl-64">
  <header className="sticky top-0 z-10 flex h-16 items-center border-b border-slate-200
    bg-white/95 px-4 backdrop-blur sm:px-6 lg:px-8">
    <button
      className="mr-3 rounded-lg p-2 text-slate-600 hover:bg-slate-100 lg:hidden"
      onClick={() => setOpen(true)}
      aria-label="Open menu"
    >
      <Menu size={21} />
    </button>

    <p className="text-sm font-semibold text-slate-700">
      Employee Management
    </p>

    <div className="ml-auto grid size-9 place-items-center rounded-full bg-teal-100
      text-sm font-bold text-teal-800">
      DA
    </div>
  </header>

  <main className="p-4 sm:p-6 lg:p-8">
    <Outlet />
  </main>
</div>
</div>
);
}
```

Explanation

This layout contains:

Sidebar

Top navbar

Main content area

`<Outlet />` displays the child route page.

43. Dashboard.tsx

File:

src/pages/Dashboard.tsx

```
import { Banknote, Users } from "lucide-react";
import { Link } from "react-router-dom";
import { PageHeader } from "../components/common/PageHeader";
import { ErrorState, LoadingState } from "../components/common/StateMessage";
import { formatCurrency } from "../lib/format";
import { useGetEmployeesQuery } from "../redux/api/employeeApi";

export default function Dashboard() {
  const { data = [], isLoading, error } = useGetEmployeesQuery();

  const totalPayroll = data.reduce(
    (sum, employee) => sum + employee.salary,
    0,
  );

  return (
    <div className="space-y-6">
      <PageHeader
        title="Dashboard"
        description="A quick overview of your employee records."
      />

      {isLoading ? (
        <LoadingState />
      ) : error ? (
        <ErrorState message="Could not connect to the Employee API." />
      ) : (
        <>
          <div className="grid gap-4 sm:grid-cols-2">
            <div className="rounded-xl border border-slate-200 bg-white p-5 shadow-sm">
              <span className="grid size-10 place-items-center rounded-lg bg-teal-100 text-teal-700">
                <Users size={20} />
              </span>

              <p className="mt-4 text-sm font-medium text-slate-500">
                Total employees
              </p>

              <p className="mt-1 text-3xl font-bold text-slate-900">
                {data.length}
              </p>
            </div>

            <div className="rounded-xl border border-slate-200 bg-white p-5 shadow-sm">
              <span className="grid size-10 place-items-center rounded-lg bg-amber-100 text-amber-700">
                <Banknote size={20} />
              </span>

              <p className="mt-4 text-sm font-medium text-slate-500">
                Total annual payroll
              </p>

              <p className="mt-1 text-3xl font-bold text-slate-900">
                {formatCurrency(totalPayroll)}
              </p>
            </div>
          </div>

          <Link
            to="/employees"
            className="inline-flex text-sm font-semibold text-teal-700 hover:text-teal-900"
          >
            Manage employee records →
          </Link>
        </>
      )}
    </div>
  );
}
```

Explanation

This page shows:

Total employees

Total payroll

Link to employees page

It uses:

`useGetEmployeesQuery()`

to load employees from backend.

44. NotFound.tsx

File:

src/pages/NotFound.tsx

```
import { Link } from "react-router-dom";
import { Button } from "../components/ui/Button";

export default function NotFound() {
  return (
    <div className="grid min-h-[60vh] place-items-center text-center">
      <div>
        <p className="text-sm font-bold text-teal-700">404</p>

        <h1 className="mt-2 text-3xl font-bold text-slate-900">
          Page not found
        </h1>

        <p className="mt-2 text-slate-500">
          The page you requested does not exist.
        </p>

        <Link to="/">
          <Button className="mt-6">Back to dashboard</Button>
        </Link>
      </div>
    </div>
  );
}
```

45. EmployeeList.tsx

File:

src/pages/employees/EmployeeList.tsx

```
import { Pencil, Plus, Search, Trash2 } from "lucide-react";
import { useMemo, useState } from "react";
import toast from "react-hot-toast";
import { Link } from "react-router-dom";
import { PageHeader } from "../../../components/common/PageHeader";
import { ErrorState, LoadingState } from "../../../components/common/StateMessage";
import { Button } from "../../../components/ui/Button";
import { getApiErrorMessage } from "../../../lib/apiError";
import { formatCurrency } from "../../../lib/format";
import {
  useDeleteEmployeeMutation,
  useGetEmployeesQuery,
} from "../../../redux/api/employeeApi";

export default function EmployeeList() {
  const [search, setSearch] = useState("");

  const { data = [], isLoading, error } = useGetEmployeesQuery();

  const [deleteEmployee, { isLoading: deleting }] =
    useDeleteEmployeeMutation();

  const filtered = useMemo(() => {
    const query = search.trim().toLowerCase();

    return query
      ? data.filter((employee) =>
          `${employee.name} ${employee.email}`
            .toLowerCase()
            .includes(query),
        )
      : data;
  }, [data, search]);

  const handleDelete = async (id: number, name: string) => {
    if (!window.confirm(`Delete ${name}? This action cannot be undone.`)) {
      return;
    }

    try {
      await deleteEmployee(id).unwrap();
      toast.success("Employee deleted");
    } catch (deleteError) {
      toast.error(getApiErrorMessage(deleteError));
    }
  };

  return (
    <div className="space-y-5">
      <PageHeader
        title="Employees"
        description="View, search, create, and update employee records."
        action={
          <Link to="/employees/new">
            <Button>
              <Plus size={17} />
              Add employee
            </Button>
          </Link>
        }
      />

      <div className="relative max-w-md">
        <Search
          className="absolute left-3 top-1/2 -translate-y-1/2 text-slate-400"
          size={18}
        />

        <input
          value={search}
          onChange={(event) => setSearch(event.target.value)}
          placeholder="Search name or email..."
          className="h-11 w-full rounded-lg border border-slate-300 bg-white pl-10 pr-4 text-sm"
        />
      </div>
    </div>
  );
}
```

```

        outline-none focus:border-teal-600 focus:ring-2 focus:ring-teal-100"
      />
    </div>

    {isLoading ? (
      <LoadingState />
    ) : error ? (
      <ErrorState message="Could not load employees. Check that the backend is running on port
        5256." />
    ) : (
      <div className="overflow-hidden rounded-xl border border-slate-200 bg-white shadow-sm">
        <div className="overflow-x-auto">
          <table className="w-full text-left text-sm">
            <thead className="border-b border-slate-200 bg-slate-50 text-xs uppercase
              tracking-wide text-slate-500">
              <tr>
                <th className="px-5 py-3">Employee</th>
                <th className="px-5 py-3">Email</th>
                <th className="px-5 py-3">Salary</th>
                <th className="px-5 py-3 text-right">Actions</th>
              </tr>
            </thead>

            <tbody className="divide-y divide-slate-100">
              {filtered.map((employee) => (
                <tr key={employee.id} className="hover:bg-slate-50">
                  <td className="px-5 py-4 font-semibold text-slate-900">
                    {employee.name}
                  </td>

                  <td className="px-5 py-4 text-slate-600">
                    {employee.email}
                  </td>

                  <td className="px-5 py-4 text-slate-600">
                    {formatCurrency(employee.salary)}
                  </td>

                  <td className="px-5 py-4">
                    <div className="flex justify-end gap-2">
                      <Link
                        to={` /employees/${employee.id}/edit`}
                        className="grid size-9 place-items-center rounded-lg border
                          border-slate-300 text-slate-600 hover:bg-slate-100"
                        aria-label={`Edit ${employee.name}`}
                      >
                        <Pencil size={16} />
                      </Link>

                      <button
                        disabled={deleting}
                        onClick={() =>
                          handleDelete(employee.id, employee.name)
                        }
                        className="grid size-9 place-items-center rounded-lg border
                          border-red-200 text-red-600 hover:bg-red-50 disabled:opacity-50"
                        aria-label={`Delete ${employee.name}`}
                      >
                        <Trash2 size={16} />
                      </button>
                    </div>
                  </td>
                </tr>
              )})}
            </tbody>
          </table>
        </div>

        {filtered.length === 0 && (
          <div className="p-10 text-center text-sm text-slate-500">
            {search
              ? "No employees match your search."
              : "No employees yet. Add your first employee."}
          </div>
        )}
      </div>
    )}
  </div>
);
}

```

Explanation

This page:

Loads employee list

Displays table

Searches employee by name/email

Navigates to create page

Navigates to edit page

Deletes employee

46. EmployeeCreate.tsx

File:

src/pages/employees/EmployeeCreate.tsx

```
import toast from "react-hot-toast";
import { useNavigate } from "react-router-dom";
import { PageHeader } from "../../components/common/PageHeader";
import { EmployeeForm } from "../../components/employee/EmployeeForm";
import { getApiErrorMessage } from "../../lib/apiError";
import { useCreateEmployeeMutation } from "../../redux/api/employeeApi";
import type { EmployeeFormValues } from "../../zod/employeeZod";
```

```
export default function EmployeeCreate() {
  const navigate = useNavigate();

  const [createEmployee, { isLoading }] = useCreateEmployeeMutation();

  const onSubmit = async (values: EmployeeFormValues) => {
    try {
      await createEmployee(values).unwrap();

      toast.success("Employee created");

      navigate("/employees");
    } catch (error) {
      toast.error(getApiErrorMessage(error));
    }
  };
};
```

```
return (
  <div className="space-y-6">
    <PageHeader
      title="New employee"
      description="Add a new employee record."
    />

    <EmployeeForm
      isSubmitting={isLoading}
      submitLabel="Create employee"
      onSubmit={onSubmit}
    />
  </div>
);
```

```
}
Create Flow
User fills form
↓
Zod validates form
↓
createEmployee(values)
↓
POST /api/Employee
↓
Backend saves employee
↓
```

Redirect to /employees

47. EmployeeEdit.tsx

File:

src/pages/employees/EmployeeEdit.tsx

```
import toast from "react-hot-toast";
import { useNavigate, useParams } from "react-router-dom";
import { PageHeader } from "../../components/common/PageHeader";
import { ErrorState, LoadingState } from "../../components/common/StateMessage";
import { EmployeeForm } from "../../components/employee/EmployeeForm";
import { getApiErrorMessage } from "../../lib/apiError";
import {
  useGetEmployeeQuery,
  useUpdateEmployeeMutation,
} from "../../redux/api/employeeApi";
import type { EmployeeFormValues } from "../../zod/employeeZod";

export default function EmployeeEdit() {
  const navigate = useNavigate();

  const id = Number(useParams().id);

  const {
    data,
    isLoading,
    error,
  } = useGetEmployeeQuery(id, {
    skip: !Number.isInteger(id),
  });

  const [updateEmployee, { isLoading: saving }] =
    useUpdateEmployeeMutation();

  const onSubmit = async (values: EmployeeFormValues) => {
    try {
      await updateEmployee({ id, ...values }).unwrap();

      toast.success("Employee updated");

      navigate("/employees");
    } catch (updateError) {
      toast.error(getApiErrorMessage(updateError));
    }
  };

  if (!Number.isInteger(id)) {
    return <ErrorState message="Invalid employee ID." />;
  }

  if (isLoading) {
    return <LoadingState />;
  }

  if (error || !data) {
    return <ErrorState message="Employee could not be found." />;
  }

  return (
    <div className="space-y-6">
      <PageHeader
        title="Edit employee"
        description={`Update ${data.name}'s employee record.`}
      />

      <EmployeeForm
        defaultValues={{
          name: data.name,
          email: data.email,
          salary: data.salary,
        }}
        isSubmitting={saving}
        submitLabel="Update employee"
        onSubmit={onSubmit}
      />
    </div>
  );
}
```

Edit Flow

Open /employees/1/edit

```
↓
useGetEmployeeQuery(1)
↓
GET /api/Employee/1
↓
Form fills with existing data
↓
User edits and saves
↓
PUT /api/Employee/1
↓
Backend updates SQL Server
```

48. index.css

File:

src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  margin: 0;
  min-width: 320px;
  min-height: 100vh;
}
```

49. Tailwind Config

File:

tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
export default {
```

content: [

```
  "./index.html",
  "./src/**/*.{js,ts,jsx,tsx}",
],
theme: {
  extend: {},
},
```

plugins: [],

```
};
```

50. Frontend Run Command

Run React frontend:

```
npm run dev
```

Vite usually opens at:

```
http://localhost:5173
```

51. Full Working Mechanism

Read Employees

React EmployeeList page opens

```
↓
useGetEmployeesQuery() runs
↓
GET http://localhost:5256/api/Employee
```

↓
EmployeeApiController.Get()
↓
EmployeeService.GetAll()
↓
_context.Employees.ToListAsync()
↓
SQL Server returns rows
↓
React displays table
Create Employee
User clicks Add employee
↓

React opens /employees/new

↓
User fills form
↓
Zod validates form
↓
createEmployee(values)
↓
POST /api/Employee
↓
Controller receives CreateEmployeeViewModel
↓
Controller maps ViewModel to Employee entity
↓
EmployeeService.Create()
↓
_context.Employees.Add(employee)
↓
SaveChangesAsync()
↓
SQL INSERT happens
↓
React redirects to employee list
Update Employee
User clicks edit icon
↓

React opens /employees/1/edit

↓
GET /api/Employee/1 loads employee
↓
Form fills with existing data
↓
User changes salary
↓
PUT /api/Employee/1
↓
Controller receives UpdateEmployeeViewModel
↓
Service finds employee
↓
Service updates values
↓
SaveChangesAsync()
↓
SQL UPDATE happens
↓
React redirects to employee list
Delete Employee
User clicks delete
↓
Browser confirm appears
↓
DELETE /api/Employee/1
↓
Controller calls service
↓
Service finds employee
↓
Service removes employee
↓
SaveChangesAsync()
↓
SQL DELETE happens

↓
RTK Query refreshes employee list

52. Why Frontend and Backend Both Validate?

Frontend validation gives fast feedback.

Backend validation protects the database.

Frontend can be bypassed using:

Postman

Swagger

DevTools

Custom script

So backend validation is always required.

Frontend validation:

zod

Backend validation:

```
[Required]  
[EmailAddress]  
[Range]
```

53. Why Use Service Layer?

Without service layer:

Controller directly talks to database

That becomes messy.

With service layer:

Controller handles HTTP

Service handles logic

DbContext handles database

This is cleaner and professional.

54. Why Use Dependency Injection?

Controller has:

```
private readonly IEmployeeService _service;
```

ASP.NET Core automatically gives EmployeeService.

Because we registered:

```
services.AddScoped<IEmployeeService, EmployeeService>();
```

This makes code flexible and testable.

55. Why Use RTK Query?

RTK Query handles:

API call

Loading state

Error state

Cache

Automatic refresh after create/update/delete

Example:

```
const { data, isLoading, error } = useGetEmployeesQuery();
```

This replaces writing manual Axios loading logic.

56. Testing Checklist

Backend

Run:

```
dotnet run
```

Open:

```
http://localhost:5256/swagger
```

Test:

```
GET /api/employee
POST /api/employee
PUT /api/employee/1
DELETE /api/employee/1
```

Database

Run in SSMS:

```
USE EmployeeApiDb;

SELECT * FROM Employees;
```

Frontend

Run:

```
npm run dev
```

Open:

```
http://localhost:5173
```

Go to:

/employees

Check browser Network tab.

Expected request:

```
GET http://localhost:5256/api/Employee
```

Expected status:

200 OK

57. Common Errors and Fixes

Error: React cannot load employees

Check backend is running:

```
http://localhost:5256/swagger
```

Check .env:

```
VITE_API_URL=http://localhost:5256
```

Restart React:

```
npm run dev
```

Error: CORS

Make sure backend has:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("ReactPolicy", policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

app.UseCors("ReactPolicy");
```

Error: Cannot connect to database

Check connection string:

```
"DefaultConnection": "Server=DESKTOP-UD9CMSS\\SQLEXPRESS;Database=EmployeeApiDb;Trusted_Connection=True;MultipleActiveResultSets=true;TrustServerCertificate=True"
```

Check SQL Server is running.

Check database exists:

```
SELECT name FROM sys.databases;
```

Error: Table does not exist

Create table:

```
USE EmployeeApiDb;
```

```
CREATE TABLE Employees
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(150) NOT NULL,
    Email NVARCHAR(255) NOT NULL,
    Salary DECIMAL(18,2) NOT NULL
);
```

Error: 404 API not found

Backend route is:

```
[Route("api/employee")]
```

Frontend should call:

/api/Employee

or:

/api/employee

Both should work in ASP.NET Core.

58. Final Summary

You built a complete Employee Management CRUD system.

The backend contains:

Model

DbContext

ViewModel

Service Interface

Service Implementation

Controller

Dependency Injection

Swagger

CORS

SQL Server connection

The frontend contains:

React Router

Redux Toolkit Query

Employee List

Employee Create

Employee Edit

Employee Delete

Search

Validation

Toast notifications

Dashboard

Reusable UI components

The complete request flow is:

React Page

↓

RTK Query API Hook

↓

ASP.NET Core Controller

↓

Employee Service

↓

ApplicationDbContext

↓

SQL Server Employees Table

This is a clean and professional structure for a beginner-to-intermediate full-stack CRUD application